JAVA INTERVIEW IMPORTANT PROGRAMS

1. Java program to Reverse Number

Question : Write a program to reverse a given number.

Algorithm :

① Read the number from user.
② Initialize rev = 0.
③ Extract last digit using num % 10.
④ Add digit to rev → rev = rev * 10 + r.
⑤ Remove last digit → num = num / 10.
⑥ Repeat steps ③ to ⑤ until num becomes 0.
⑦ Print rev.

Code :

```
import java.util.*;
public class ReverseNumber {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter any number : ");
    int num = sc.nextInt();
    int rev = 0, r;
    while(num != 0) {
      r = num % 10;
      rev = rev * 10 + r;
      num = num / 10;
    }
    System.out.println("Reverse : " + rev);
  }
}
```

Sample Input : 12345
Output : 54321

Time Complexity : O(d) where d = number of digits
Space Complexity : O(1)

2. Java program to find Palindrome Number

Question : Write a program to check whether a number is palindrome or not.

Algorithm :

① Read the number from user.
② Store the number in another variable (originalNumber).
③ Reverse the number (same as previous program).
④ Compare originalNumber and rev.
⑤ If equal → palindrome
else → not palindrome.

Code :

```
import java.util.*;
public class PalindromeNumber {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter any number : ");
    int num = sc.nextInt();
    int originalNumber = num;
    int rev = 0, r;
    while(num != 0) {
      r = num % 10;
      rev = rev * 10 + r;
      num = num / 10;
    }
    if(originalNumber == rev)
      System.out.println("Number is Palindrome");
    else
      System.out.println("Number is not Palindrome");
  }
}
```

Sample Input : 121
Output : Number is Palindrome

Time Complexity : O(d) where d = number of digits
Space Complexity : O(1)

Note :

- Any number which reads same from left to right and right to left is called Palindrome.
- Example Palindrome numbers : 121, 12321, 7, 1001
- Example non-palindrome numbers : 123, 4567, 789

JAVA INTERVIEW IMPORTANT PROGRAMS

3. Java program to find Prime Number

Question : Write a program to check whether a number is prime or not.

Algorithm :

① Read the number from user.
② If number <= 1 → not prime.
③ Check divisibility from 2 to sqrt(n).
④ If number is divisible by any i, then not prime.
⑤ If not divisible by any i → prime.

Code :

```
import java.util.*;
public class PrimeNumber {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter any number : ");
    int n = sc.nextInt();
    boolean isPrime = true;
    if (n <= 1) {
       isPrime = false;
    } else {
       for (int i = 2; i * i <= n; i++) {
           if (n % i == 0) {
              isPrime = false;
              break;
           }
       }
    }
    if (isPrime)
       System.out.println(n + " is a Prime");
    else
       System.out.println(n + " is not a Prime");
  }
}
```

Sample Input : 17
Output : 17 is a Prime

Time Complexity : $O(\sqrt{n})$
Space Complexity : O(1)

Note :
- Prime numbers are numbers greater than 1 which have exactly two factors : 1 and itself.
- Example Prime numbers : 2, 3, 5, 7, 11, 13, 17, ...

4. Java program to find Armstrong Number

Question : Write a program to check whether a number is Armstrong or not.

Algorithm :

① Read the number from user.
② Count total number of digits in the number.
③ For each digit, calculate (digit ^ count) and add to sum.
④ If sum == original number → Armstrong
else → Not Armstrong.

Code :

```
import java.util.*;
public class ArmstrongNumber {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter any number : ");
    int n = sc.nextInt();
    int temp = n, sum = 0, count = 0;
    // Count total number of digits
    while (temp > 0) {
       temp = temp / 10;
       count++;
    }
    // Calculate sum of (digit ^ count)
    temp = n;
    while (temp > 0) {
       int digit = temp % 10;
       sum = sum + (int)Math.pow(digit, count);
       temp = temp / 10;
    }
    if (sum == n)
       System.out.println(n + " is Armstrong number");
    else
      System.out.println(n + " is not Armstrong number");
  }
}
```

Sample Input : 153
Output : 153 is Armstrong number

Time Complexity : O(d) where d = number of digits
Space Complexity : O(1)

Note :
- Armstrong numbers are numbers that are equal to the sum of their own digits each raised to the power of total digits.
- Example Armstrong numbers : 0, 1, 153, 370, 371, 407, ...

JAVA INTERVIEW IMPORTANT PROGRAMS

5. Java program to Find Odd or Even number

Question : Write a program to check whether a number is Odd or Even.

Algorithm :

① Read the number from user.
② If number % 2 === 0 → Even
③ Else → Odd.
④ Print the result.

Code :

```
import java.util.*;
public class OddEven {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter any number : ");
      int num = sc.nextInt();
      if (num % 2 === 0)
         System.out.println(num + " is Even");
      else
         System.out.println(num + " is Odd");
   }
}
```

Sample Input : 24
Output : 24 is Even

Time Complexity : O(1)
Space Complexity : O(1)

Note :
- If number is divisible by 2, it is Even.
- Otherwise, it is Odd.
- 0 is considered as Even.

6. Java program to swap two numbers without using third variable

Question : Write a program to swap two numbers without using third variable.

Algorithm :

① Read two numbers a and b.
② Use arithmetic operations to swap.
(a = a + b, b = a - b, a = a - b)
③ Print the swapped numbers.

Code :

```
import java.util.*;
public class SwapNumbers {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first number : ");
      int a = sc.nextInt();
      System.out.print("Enter second number : ");
      int b = sc.nextInt();
      a = a + b;    // Step 1
      b = a - b;    // Step 2
      a = a - b;    // Step 3
      System.out.println("After swapping :");
      System.out.println("a = " + a + ", b = " + b);
   }
}
```

Sample Input : a = 10, b = 25
Output : a = 25, b = 10

Time Complexity : O(1)
Space Complexity : O(1)

Note :
- This method works only for integers.
- For very large numbers, (a + b) might exceed the integer range.
- XOR swap can also be used.

JAVA INTERVIEW IMPORTANT PROGRAMS

7. Java program to find Fibonacci series upto a given number of terms

Question : Write a program to print Fibonacci series upto n terms.

Algorithm :

① Read number of terms n from user.
② Initialize first = 0, second = 1.
③ Print first two terms.
④ For i = 3 to n
    next = first + second
    Print next
    first = second
    second = next
⑤ Repeat step 4 until i > n.

Code :

```
import java.util.*;
public class FibonacciSeries {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter number of terms : ");
      int n = sc.nextInt();
      int first = 0, second = 1, next;
      System.out.print(first + " " + second + " ");
      for (int i = 3; i <= n; i++) {
          next = first + second;
          System.out.print(next + " ");
          first = second;
          second = next;
      }
   }
}
```

Sample Input : 7
Output : 0 1 1 2 3 5 8

Time Complexity : O(n)
Space Complexity : O(1)

Note :
- Fibonacci series starts with 0 and 1.
- Each next term is sum of previous two terms.

8. Java program to find Factorial of a given number

Question : Write a program to find factorial of a given number.

Algorithm :

① Read number n from user.
② Initialize fact = 1.
③ For i = 1 to n
    fact = fact * i
④ Print fact.

Code :

```
import java.util.*;
public class FactorialNumber {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a number : ");
      int n = sc.nextInt();
      long fact = 1;
      for (int i = 1; i <= n; i++) {
          fact = fact * i;
      }
      System.out.println("Factorial of " + n + " is " + fact);
   }
}
```

Sample Input : 5
Output : Factorial of 5 is 120

Time Complexity : O(n)
Space Complexity : O(1)

Note :
- Factorial of 0 is 1.
- Factorial grows very fast, use long for bigger numbers.

JAVA INTERVIEW IMPORTANT PROGRAMS

9. Java program to find Sum of digits of a number

Question : Write a program to calculate the sum of digits of a number.

Algorithm :

① Read the number from user.
② Initialize sum = 0.
③ Extract last digit using num % 10.
④ Add digit to sum ⟶ sum = sum + digit.
⑤ Remove last digit ⟶ num = num / 10.
⑥ Repeat step 3 to 5 until num becomes 0.
⑦ Print sum.

Code :

```
import java.util.*;
public class SumOfDigits {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter any number : ");
      int num = sc.nextInt();
      int sum = 0;
      while (num > 0) {
          int digit = num % 10;
          sum = sum + digit;
          num = num / 10;
      }
      System.out.println("Sum of digits is : " + sum);
   }
}
```

```
Sample Input  :  12345
Output        :  15
```

Time Complexity : O(d) where d = number of digits
Space Complexity : O(1)

> **Note :**
> Extract last digit using % 10 and reduce number using / 10.

10. Java program to find Number of digits in a number

Question : Write a program to count the number of digits in a given number.

Algorithm :

① Read the number from user.
② Initialize count = 0.
③ Divide the number by 10.
④ Increase count by 1.
⑤ Repeat step 3 to 4 until num becomes 0.
⑥ Print count.

Code :

```
import java.util.*;
public class CountDigits {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter any number : ");
      int num = sc.nextInt();
      int count = 0;
      while (num > 0) {
          num = num / 10;
          count++;
      }
      System.out.println("Number of digits is : " + count);
   }
}
```

```
Sample Input  :  987654321
Output        :  9
```

Time Complexity : O(d) where d = number of digits
Space Complexity : O(1)

> **Note :**
> Every time we divide the number by 10, one digit is removed.

JAVA INTERVIEW IMPORTANT PROGRAMS

11. Java program to reverse a string

Question : Write a program to reverse a given string.

Algorithm :

① Read the string from user.
② Traverse from end to start.
③ Append each character to result.
④ Print the reversed string.

Code :

```
import java.util.*;
public class ReverseString {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string : ");
      String str = sc.nextLine();
      String rev = "";
      for (int i = str.length() - 1; i >= 0; i--){
          rev = rev + str.charAt(i);
       }
      System.out.println("Reversed String : " + rev);
   }
}
```

Sample Input : hello
Output : olleh

Time Complexity : O(n)
Space Complexity : O(1)

Note :
We are traversing the string once from end to start.

12. Java program to reverse each word of a given string

Question : Write a program to reverse each word of a given string.

Algorithm :

① Read the string from user.
② Split the string into words.
③ Reverse each word individually.
④ Print the words in reversed form.

Code :

```
import java.util.*;
public class ReverseEachWord {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string : ");
      String str = sc.nextLine();
      String[] words = str.split(" ");
      String result = "";
      for (String word : words) {
          String rev = "";
          for (int i = word.length() - 1; i >= 0; i--) {
              rev = rev + word.charAt(i);
          }
          result = result + rev + " ";
      }
      System.out.println("Result : " + result.trim());
   }
}
```

Sample Input : Java is good
Output : avaJ si doog

Time Complexity : O(n)
Space Complexity : O(n)

Note :
We split the string into words and reverse each word separately.

JAVA INTERVIEW IMPORTANT PROGRAMS

13. Java program to find String Palindrome

Question : Write a program to check whether a string is palindrome or not.

Algorithm :

① Read the string from user.
② Compare characters from start and end moving towards center.
③ If all characters match → palindrome.
④ Else → not palindrome.

Code :

```
import java.util.*;
public class StringPalindrome {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a string : ");
    String str = sc.nextLine();
    String rev = "";
    for (int i = str.length() - 1; i >= 0; i--) {
       rev = rev + str.charAt(i);
    }
    if (str.equalsIgnoreCase(rev))
       System.out.println("String is Palindrome");
    else
       System.out.println("String is not Palindrome");
  }
}
```

Sample Input : madam
Output : String is Palindrome

Time Complexity : O(n)
Space Complexity : O(n)

Note :
- We can also compare without using extra space by two-pointer approach.
- equalsIgnoreCase() ignores case.

14. Java program to check Two Strings are Anagrams or not

Question : Write a program to check whether two strings are anagrams or not.

Algorithm :

① Read two strings from user.
② If lengths are different → not anagram.
③ Sort characters of both strings.
④ If sorted strings are equal → anagram.
⑤ Else → not anagram.

Code :

```
import java.util.*;
public class AnagramString {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter first string : ");
    String str1 = sc.nextLine();
    System.out.print("Enter second string : ");
    String str2 = sc.nextLine();
    if (str1.length() != str2.length()) {
       System.out.println("Not Anagrams");
       return;
    }
    char ch1[] = str1.toCharArray();
    char ch2[] = str2.toCharArray();
    Arrays.sort(ch1);
    Arrays.sort(ch2);
    if (Arrays.equals(ch1, ch2))
       System.out.println("Strings are Anagrams");
    else
       System.out.println("Strings are not Anagramsy");
  }
}
```

Sample Input : listen silent
Output : Strings are Anagrams

Time Complexity : O(n log n)
Space Complexity : O(n)

Note :
- Anagrams must have same length.
- Sorting both strings and comparing is the most common approach.

JAVA INTERVIEW IMPORTANT PROGRAMS

15. Java program to count occurrence of a character in a string

Question : Write a program to count the occurrence of a given character in a string.

Algorithm :

① Read string and character from user.
② Initialize count = 0.
③ Traverse the string.
④ If character matches, increment count.
⑤ Print count.

Code :

```
import java.util.*;
public class CountChar {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string : ");
      String str = sc.nextLine();
      System.out.print("Enter a character : ");
      char ch = sc.next().charAt(0);
      int count = 0;
      for (int i = 0; i < str.length(); i++) {
         if (str.charAt(i) == ch) {
            count++;
         }
      }
      System.out.println("Occurrence of '" + ch + "'
            + " is : " + count);
   }
}
```

```
Sample Input  : Hello World , o
Output        : 2
```

Time Complexity : O(n)
Space Complexity : O(1)

Note :
- We traverse the string once.
- Compare each character with given character.

16. Java program to count number of words in a string

Question : Write a program to count the number of words in a string.

Algorithm :

① Read string from user.
② Trim leading and trailing spaces.
③ Split the string by one or more spaces.
④ Count the words.
⑤ Print count.

Code :

```
import java.util.*;
public class CountWords {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a string : ");
      String str = sc.nextLine().trim();
      if (str.length() == 0) {
         System.out.println("No words");
         return;
      }
      String[] words = str.split("\\s+");   // one or
                                            // more spaces
      int count = words.length;
      System.out.println("Number of words : " + count);
   }
}
```

```
Sample Input  : Java is awesome
Output        : 3
```

Time Complexity : O(n)
Space Complexity : O(n)

Note :
- trim() removes extra spaces at start and end.
- split("\\s+") splits by one or more spaces.

JAVA INTERVIEW IMPORTANT PROGRAMS

17. Java program to find LCM of two numbers

Question : Write a program to find LCM of two numbers.

Algorithm :

① Read two numbers a and b.

② Find greater number.

③ Initialize lcm = greater.

④ While (true)

```
if (lcm % a == 0 && lcm % b == 0)
    break;
else
    lcm++;
```

⑤ Print lcm.

Code :

```
import java.util.*;
public class LCM {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first number : ");
      int a = sc.nextInt();
      System.out.print("Enter second number : ");
      int b = sc.nextInt();
      int greater = (a > b) ? a : b;
      int lcm = greater;
      while (true) {
          if (lcm % a == 0 && lcm % b == 0)
              break;
          lcm++;
      }
      System.out.println("LCM is : " + lcm);
   }
}
```

```
Sample Input  :  12 18
Output        :  36
```

Time Complexity : O(n)
Space Complexity : O(1)

Note :
LCM can also be calculated as
LCM(a,b) = (a * b) / GCD(a,b)
(Efficient approach).

18. Java program to find HCF (GCD) of two numbers

Question : Write a program to find HCF (GCD) of two numbers.

Algorithm :

① Read two numbers a and b.

② While (b != 0)

```
rem = a % b
a = b
b = rem
```

③ Print a (which is HCF).

Code :

```
import java.util.*;
public class HCF {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first number : ");
      int a = sc.nextInt();
      System.out.print("Enter second number : ");
      int b = sc.nextInt();
      while (b != 0) {
          int rem = a % b;
          a = b;
          b = rem;
      }
      System.out.println("HCF (GCD) is : " + a);
   }
}
```

```
Sample Input  :  48 18
Output        :  6
```

Time Complexity : O(log n)
Space Complexity : O(1)

Note :
This is Euclidean Algorithm.
Very efficient for large numbers.